

RELATÓRIO COMPARATIVO — ANÁLISE AP01

Laboratório de Programação — IFCE Campus Maracanaú
Comparação entre as soluções desenvolvidas pelo aluno e as soluções produzidas pelo ChatGPT para as quatro questões da Análise AP01 (21/09/2026).

IA adotada: ChatGPT (GPT-5.6 Luna).

O relatório segue a proposta da atividade: comparar as soluções, verificar correção e restrições, identificar situações de falha e registrar o aprendizado obtido com a comparação.

1. Objetivo e metodologia

A análise não busca definir se uma solução é simplesmente “melhor” que a outra. O foco é identificar diferenças de abordagem, verificar se cada programa atende ao enunciado e observar aspectos de eficiência, clareza, robustez e aderência às restrições.

Para cada questão, foram considerados: (1) funcionamento em relação ao enunciado; (2) principais diferenças de lógica; (3) memória e quantidade de processamento; (4) casos em que a solução pode falhar; e (5) aprendizado que pode ser extraído da comparação.

Observação importante: os códigos do aluno foram analisados como enviados, sem alterações. A atividade determina que as soluções feitas em sala não sejam modificadas posteriormente.

Questão 1 — Esteganografia

Código do aluno

```
#include <stdio.h>

int main() {
    unsigned char M;

    printf("Digite um número inteiro (0 a 255): ");
    scanf("%hhu", &M);

    unsigned char R = (M >> 2) & 1;
    unsigned char G = (M >> 1) & 1;
    unsigned char B = (M >> 0) & 1;

    printf ("R = %hhu \n", R);
    printf ("G = %hhu \n", G);
    printf ("B = %hhu \n", B);

    return 0;
}
```

Código produzido pela IA

```
#include <stdio.h>

int main() {
    int R, G, B, M;

    scanf("%d %d %d", &R, &G, &B);
    scanf("%d", &M);

    R = (R & ~1) | ((M >> 2) & 1);
    G = (G & ~1) | ((M >> 1) & 1);
    B = (B & ~1) | (M & 1);

    printf("%d %d %d\n", R, G, B);

    return 0;
}
```

Comparação direta lado a lado

Seu código	Código da IA
<pre>#include <stdio.h> int main() { unsigned char M; printf("Digite um número inteiro (0 a 255): "); scanf("%hhu", &M); unsigned char R = (M >> 2) & 1; unsigned char G = (M >> 1) & 1; unsigned char B = (M >> 0) & 1; printf ("R = %hhu \n", R); printf ("G = %hhu \n", G); printf ("B = %hhu \n", B); return 0; }</pre>	<pre>#include <stdio.h> int main() { int R, G, B, M; scanf("%d %d %d", &R, &G, &B); scanf("%d", &M); R = (R & ~1) ((M >> 2) & 1); G = (G & ~1) ((M >> 1) & 1); B = (B & ~1) (M & 1); printf("%d %d %d\n", R, G, B); return 0; }</pre>

Leitura do problema

O código do aluno lê apenas M e extrai seus três bits, produzindo R, G e B como 0 ou 1. Porém, o enunciado define R, G e B como valores de uma cor já existentes e pede que os bits da mensagem sejam armazenados nos bits menos significativos desses três valores.

Diferença principal

A solução do aluno trata R, G e B como os próprios bits da mensagem. A solução da IA preserva os demais bits de cada componente e substitui somente o bit 0. Isso corresponde diretamente à operação de esconder a mensagem em uma cor.

Correção

Para o enunciado fornecido, a solução da IA atende à operação descrita. A solução do aluno não realiza a etapa de receber/preservar os valores originais de R, G e B; portanto, não implementa completamente a esteganografia pedida.

Eficiência

As duas soluções usam quantidade constante de memória e poucas operações. A diferença de eficiência é pequena; o ponto decisivo é a aderência ao enunciado.

Possíveis falhas

Na solução do aluno, qualquer entrada R/G/B original seria ignorada porque nem sequer é lida. Na solução da IA, assume-se entrada válida nos intervalos esperados; não há validação explícita de 0-255.

Aprendizado

A comparação mostra que extrair bits de M é apenas uma parte do problema. Em esteganografia, é necessário preservar a informação existente e alterar somente os bits especificados.

Questão 2 — Média móvel

Código do aluno

```
#include <stdio.h>

int main() {

    int n = 10;
    int soma = 0;
    int xi_1, xi, xi1;

    for(int i = 2; i <= n; i++) {
        printf("Informe o valor de x%d x%d e x%d: ", i-2, i-1, i);
        scanf("%d %d %d", &xi_1, &xi, &xi1);
        soma = xi_1 + xi + xi1;
        float y = soma / 3.0;
        printf("O valor de y%d é %.2f\n", i-1, y);
    }

    return 0;
}
```

Código produzido pela IA

```
#include <stdio.h>

int main() {
    int n;
    int anterior, atual, proximo;
    int i;

    scanf("%d", &n);

    if (n < 3) {
        return 0;
    }

    scanf("%d", &anterior);
    scanf("%d", &atual);

    for (i = 2; i < n; i++) {
        scanf("%d", &proximo);

        printf("%.2f", (anterior + atual + proximo) / 3.0);

        if (i < n - 1) {
            printf(" ");
        }

        anterior = atual;
        atual = proximo;
    }

    printf("\n");
    return 0;
}
```

Comparação direta lado a lado

Seu código	Código da IA
<pre>#include <stdio.h> int main() { int n = 10; int soma = 0; int xi_1, xi, xi1; for(int i = 2; i <= n; i++) { printf("Informe o valor de x%d x%d e x%d: ", i-2, i-1, i); scanf("%d %d %d", &xi_1, &xi, &xi1); soma = xi_1 + xi + xi1; float y = soma / 3.0; printf("O valor de y%d é %.2f\n", i-1, y); } return 0; }</pre>	<pre>#include <stdio.h> int main() { int n; int anterior, atual, proximo; int i; scanf("%d", &n); if (n < 3) { return 0; } scanf("%d", &anterior); scanf("%d", &atual); for (i = 2; i < n; i++) { scanf("%d", &proximo); printf("%.2f", (anterior + atual + proximo) / 3.0); if (i < n - 1) { printf(" "); } anterior = atual; atual = proximo; } printf("\n"); return 0; }</pre>

Leitura do problema

O código do aluno fixa $n = 10$ e, em cada iteração, solicita três valores de uma vez. A solução da IA lê n e processa a sequência mantendo somente três amostras consecutivas.

Diferença principal

A solução do aluno calcula médias para conjuntos de três valores digitados separadamente. A solução da IA trata os dados como uma única sequência e desloca a janela: anterior, atual e próximo.

Correção

A solução da IA segue a definição de média móvel centrada e produz y_2 até $y_{(n-1)}$. O código do aluno pode produzir médias para os pares de janelas informados, mas não representa corretamente uma entrada geral de n amostras porque fixa $n = 10$ e não reaproveita as amostras entre janelas.

Eficiência

Ambas usam memória constante. A solução da IA lê cada amostra da sequência uma vez e reutiliza as duas últimas amostras, evitando entradas repetidas.

Possíveis falhas

O aluno não consegue adaptar n sem alterar o código e pode calcular resultados diferentes do que seria esperado quando a sequência é fornecida como uma única lista. A solução da IA trata $n < 3$, mas não valida falhas de leitura.

Aprendizado

A ideia central é uma janela deslizante: para calcular a próxima média não é necessário guardar toda a sequência, apenas as amostras necessárias.

Questão 3 — Min-Max Scaling

Código do aluno

```
#include <stdio.h>

int main() {
    int n = 5;
    unsigned char x, intensidade, x_min, x_max;

    printf("Informe a intensidade de X (0 a 255): ");
    scanf("%hhu", &x);

    for (int i = 1; i <= n; i++) {
        printf("Informe a intensidade de X%d (0 a 255): ", i);
        scanf("%hhu", &intensidade);

        if (intensidade < x_min) {
            x_min = intensidade;
        }

        if (intensidade > x_max) {
            x_max = intensidade;
        }
    }

    unsigned char x_normalizado = (x - x_min) / (x_max - x_min);
    printf("X' = %hhu \n", x_normalizado);
}
```

Código produzido pela IA

```
#include <stdio.h>

int main() {
    int n;
    int x;
    int xmin, xmax;
    int i;
    double normalizado;

    scanf("%d", &n);
    scanf("%d", &x);

    xmin = x;
    xmax = x;

    for (i = 1; i < n; i++) {
        scanf("%d", &x);

        if (x < xmin) xmin = x;
        if (x > xmax) xmax = x;
    }

    scanf("%d", &x);

    if (xmin == xmax)
        normalizado = 0.0;
    else
        normalizado = (double)(x - xmin) / (xmax - xmin);

    printf("Minimo = %d\n", xmin);
    printf("Maximo = %d\n", xmax);
    printf("Normalizado = %.6f\n", normalizado);

    return 0;
}
```

Comparação direta lado a lado

Seu código	Código da IA
-------------------	---------------------

```
#include <stdio.h>
```

```
int main() {
    int n = 5;
    unsigned char x, intensidade, x_min, x_max;

    printf("Informe a intensidade de X (0 a 255): ");
    scanf("%hhu", &x);

    for (int i = 1; i <= n; i++) {
        printf("Informe a intensidade de X%d (0 a 255): ", i);
        scanf("%hhu", &intensidade);

        if (intensidade < x_min) {
            x_min = intensidade;
        }

        if (intensidade > x_max) {
            x_max = intensidade;
        }
    }

    unsigned char x_normalizado = (x - x_min) / (x_max - x_min);
    printf("X' = %hhu \n", x_normalizado);
}
```

```
#include <stdio.h>
```

```
int main() {
    int n;
    int x;
    int xmin, xmax;
    int i;
    double normalizado;

    scanf("%d", &n);
    scanf("%d", &x);

    xmin = x;
    xmax = x;

    for (i = 1; i < n; i++) {
        scanf("%d", &x);

        if (x < xmin) xmin = x;
        if (x > xmax) xmax = x;
    }

    scanf("%d", &x);

    if (xmin == xmax)
        normalizado = 0.0;
    else
        normalizado = (double)(x - xmin) / (xmax - xmin);

    printf("Minimo = %d\n", xmin);
    printf("Maximo = %d\n", xmax);
    printf("Normalizado = %.6f\n", normalizado);

    return 0;
}
```

Leitura do problema

O código do aluno declara `x`, `intensidade`, `x_min` e `x_max`, mas `x_min` e `x_max` não são inicializados antes das comparações. Além disso, lê uma quantidade fixa de cinco intensidades após ler `x`.

Diferença principal

A solução da IA inicializa mínimo e máximo com a primeira intensidade efetivamente pertencente à sequência e depois percorre as demais. Também usa `double` para preservar a parte fracionária da normalização.

Correção

A solução do aluno apresenta problemas importantes: `x_min` e `x_max` são usados antes de receber valores definidos; `x` é lido separadamente da sequência; e o cálculo em `unsigned char` faz a divisão de inteiros e depois armazena o resultado em um tipo inteiro, perdendo a parte decimal. A solução da IA trata esses pontos.

Eficiência

As duas abordagens podem usar memória constante. A solução da IA também faz uma única passagem para encontrar mínimo e máximo.

Possíveis falhas

A solução da IA trata o caso `xmin == xmax` para evitar divisão por zero. O enunciado não explicita o que fazer nesse caso; portanto, 0.0 é uma decisão de implementação e deve ser apresentada como tal.

Aprendizado

Variáveis de mínimo e máximo precisam começar com valores válidos. Além disso, normalização em `[0,1]` exige tipo com parte fracionária; usar `unsigned char` impede representar, por exemplo, 0.5.

Questão 4 — Remoção de break e continue

Código do aluno

```
#include <stdio.h>

int main() {
    int numero, soma = 1, contador = -1;

    do {
        scanf("%d", &numero);
        soma += numero;
        contador++;
    } while (numero != -1);

    printf("Soma = %d \n", soma);
    printf("Quantidade = %d \n", contador);

    return 0;
}
```

Código produzido pela IA

```
#include <stdio.h>

int main() {
    int numero;
    int soma = 0;
    int contador = 0;

    scanf("%d", &numero);

    while (numero != -1) {
        if (numero >= 0 && numero <= 255) {
            soma += numero;
            contador++;
        }
        scanf("%d", &numero);
    }

    printf("Soma = %d\n", soma);
    printf("Quantidade = %d\n", contador);

    return 0;
}
```

Comparação direta lado a lado

Seu código	Código da IA
<pre>#include <stdio.h> int main() { int numero, soma = 1, contador = -1; do { scanf("%d", &numero); soma += numero; contador++; } while (numero != -1); printf("Soma = %d \n", soma); printf("Quantidade = %d \n", contador); return 0; }</pre>	<pre>#include <stdio.h> int main() { int numero; int soma = 0; int contador = 0; scanf("%d", &numero); while (numero != -1) { if (numero >= 0 && numero <= 255) { soma += numero; contador++; } scanf("%d", &numero); } printf("Soma = %d\n", soma); printf("Quantidade = %d\n", contador); return 0; }</pre>

Leitura do problema

O código do aluno usa do-while, mas inicializa soma = 1 e contador = -1 e soma/conta também o valor -1 antes de encerrar. O programa original, por outro lado, encerra antes de somar/contar -1 e ignora valores fora de 0 a 255.

Diferença principal

A solução da IA controla a condição de continuidade diretamente no while e usa uma condição para processar apenas números entre 0 e 255. Assim, não precisa de break nem continue.

Correção

A solução da IA reproduz o comportamento descrito no programa original: -1 encerra; números negativos diferentes de -1 e números acima de 255 são ignorados; valores de 0 a 255 são somados e contabilizados. O código do aluno não mantém esse comportamento.

Eficiência

Ambos usam memória constante. A solução da IA utiliza uma leitura inicial e depois atualiza o número ao final de cada iteração.

Possíveis falhas

O código do aluno altera os resultados mesmo em uma entrada simples porque começa com soma e contador incompatíveis com o programa original e inclui o -1. A solução da IA depende de entrada numérica válida, assim como o programa original.

Aprendizado

Para remover break e continue mantendo comportamento, é importante transformar as condições de controle em condições explícitas do laço e da execução do corpo.

6. Síntese das principais diferenças

Q uestão	Seu código	Código da IA	Ponto crítico
1	Extraí os três bits de M e os imprime como R, G e B.	Substitui apenas o bit menos significativo de R, G e B, preservando os demais bits.	Aderência ao conceito de esconder a mensagem em uma cor.
2	Fixa n=10 e lê três valores por vez.	Lê n e usa janela deslizante com três amostras.	Representação correta da sequência e reutilização das amostras.
3	Usa n=5, não inicializa min/max e armazena normalização em inteiro.	Inicializa min/max corretamente e usa double na normalização.	Inicialização e preservação da parte decimal.
4	do-while com soma/contador iniciais incompatíveis e inclui -1.	while condicionado por numero != -1 e valida faixa 0-255.	Manter exatamente o comportamento original sem break/continue.

7. O que foi aprendido com a comparação

A comparação evidencia que uma solução pode parecer funcionar em exemplos simples e ainda assim não cumprir integralmente o enunciado. Na questão 1, a diferença entre extrair bits e inserir bits em componentes existentes é fundamental. Na questão 2, a janela móvel exige reutilização das amostras da sequência. Na questão 3, inicialização correta e tipos numéricos são essenciais para preservar o resultado decimal. Na questão 4, equivalência de comportamento exige analisar exatamente quando cada valor é processado ou descartado.

Se as questões fossem resolvidas novamente, uma estratégia mais segura seria primeiro traduzir o enunciado em entradas, saídas, restrições e casos-limite; depois implementar; e, por fim, testar manualmente exemplos que envolvam limites, valores repetidos e condições de parada.

8. Resposta às perguntas finais da atividade

O uso da Inteligência Artificial modificou sua compreensão sobre alguma das questões da prova?
Sim. A comparação pode modificar a compreensão principalmente ao mostrar que pequenas diferenças na interpretação do enunciado alteram completamente a solução. A questão 1 evidencia a diferença entre obter bits de uma mensagem e realmente incorporá-los aos pixels; a questão 2 mostra a importância de uma janela deslizante; a questão 3 evidencia como tipos e inicialização afetam a correção; e a questão 4 mostra como preservar exatamente o comportamento de um programa ao remover comandos de controle.

Conclusão
A análise reforça que o código produzido por uma IA precisa ser verificado contra o enunciado. O resultado da IA não deve ser aceito automaticamente: é necessário testar a lógica, conferir as restrições e identificar casos em que a implementação pode falhar. Essa comparação também ajuda a transformar erros de implementação em aprendizados concretos para futuras questões.

Observação para a entrega: o enunciado solicita que todos os códigos estejam no GitHub e que o link do repositório seja encaminhado no Classroom. Este relatório não inventa esse link, pois ele não foi fornecido nesta conversa.